Codebreakers: ingeniería inversa de un ejecutable ELF mediante análisis estático y dinámico

Juan Carlos Bohorquez y Julian Eduardo Arenas
Programa de Ingeniería de Sistemas, Escuela Colombiana de Ingeniería Julio Garavito, Bogotá, Colombia
Fundamentos de Seguridad Informática — Laboratorio 6 · Eciciencia 2026

***Resumen*—Este informe documenta la resolución del reto tipo CTF «Codebreakers», cuyo objetivo era recuperar una contraseña oculta en un ejecutable ELF de 64 bits, sin acceso a su código fuente, y obtener la bandera de validación. Se combinó análisis estático con Ghidra y análisis dinámico con ltrace sobre una máquina virtual Kali Linux. El análisis estático permitió ubicar la función `main`, el mensaje de error y una máscara XOR (`ECI_VECTOR_2026!`) que resultó no ser la clave. Como la clave se calculaba en tiempo de ejecución, se interceptó la llamada a `strcmp` con ltrace, que la expuso en texto plano (`BREAKER#2026`) y permitió obtener la bandera `FLAG{CODEBREAKERS_ECI}`. Los resultados muestran que comparar secretos con funciones estándar sobre texto plano anula la protección que aportan las rutinas criptográficas del programa.**

***Términos para indexación*—análisis dinámico, análisis estático, CTF, Ghidra, ingeniería inversa, ltrace, ofuscación, strcmp.**

I. INTRODUCCIÓN

En seguridad informática, la ingeniería inversa permite comprender el comportamiento de un programa sin disponer de su código fuente. Este informe presenta la resolución de «Codebreakers», un reto tipo CTF (*Capture The Flag*) del Laboratorio 6 de Fundamentos de Seguridad Informática. El reto entrega un ejecutable ELF de 64 bits, `reto_codebreakers_linux_x64`, que solicita una clave por consola y, si es correcta, muestra la bandera de validación.

El objetivo fue descubrir la clave oculta y obtener la bandera. Para ello se combinaron dos enfoques: el análisis estático, con el que se comprendió la lógica del programa, y el análisis dinámico, con el que se extrajo la clave durante la ejecución. El documento describe el entorno (Sección II), el análisis estático (III), el análisis dinámico (IV), los resultados (V) y las conclusiones (VI).

II. ENTORNO EXPERIMENTAL

Para analizar el binario de forma segura se empleó un entorno híbrido. El análisis estático se realizó en Windows 10/11 con JDK 25, necesario para ejecutar Ghidra [1]. La ejecución y la depuración se llevaron a cabo en una máquina virtual con Kali Linux [2] (x86-64) sobre VMware, de modo que el binario desconocido se ejecutara de forma aislada. La Tabla I resume los componentes utilizados.

TABLA I
COMPONENTES DEL ENTORNO DE LABORATORIO

Componente	Descripción
Análisis estático	Windows 10/11 con JDK 25
Ingeniería inversa	Ghidra (NSA): desensamblado y decompilación [1]
Análisis dinámico	Kali Linux x86-64 en VMware [2]
Depuración	`ltrace`: trazado de llamadas a bibliotecas [3]
Binario objetivo	`reto_codebreakers_linux_x64` (formato ELF)

III. ANÁLISIS ESTÁTICO

A. Búsqueda de cadenas

El archivo ELF se importó en un nuevo proyecto de Ghidra. El primer paso fue buscar cadenas de texto para hallar puntos de referencia en la estructura del binario. Se identificaron dos cadenas fundamentales: la estructura de la bandera esperada, `FLAG{CODEBREAKERS_ECI}` (Fig. 1), y el mensaje de error «ACCESO DENEGADO» (Fig. 2). Aunque la bandera figura como cadena literal, el programa solo la imprime tras validar la clave (Fig. 4), de modo que el verdadero objetivo era descubrir esa clave.

B. Localización de la función de validación

Al seguir las referencias cruzadas (XREF) de la cadena «ACCESO DENEGADO», Ghidra condujo directamente a la función `main`, donde ocurre la validación (Fig. 3). La referencia corresponde a un salto condicional (`JNZ`) hacia la rama de rechazo.

C. Decompilación y análisis lógico

La vista *Decompiler* de Ghidra permitió reconstruir el código C de `main` (Fig. 4). En él aparecen dos constantes de 64 bits, `0x544345565f494345` y `0x21363230325f524f`, que, leídas en orden *little-endian*, equivalen a los bytes ASCII «ECI_VECT» y «OR_2026!»; juntas forman la cadena `ECI_VECTOR_2026!`.

Esa cadena no es la contraseña final, sino una máscara XOR. Según el flujo identificado, el programa descifra un valor en memoria mediante un algoritmo AES-256, le aplica una operación XOR (`^`) con la máscara y compara el resultado con la entrada del usuario mediante `strcmp` [4]. Si la comparación devuelve cero, imprime «ACCESO CONCEDIDO» y la

bandera; en caso contrario, imprime «ACCESO DENEGADO».

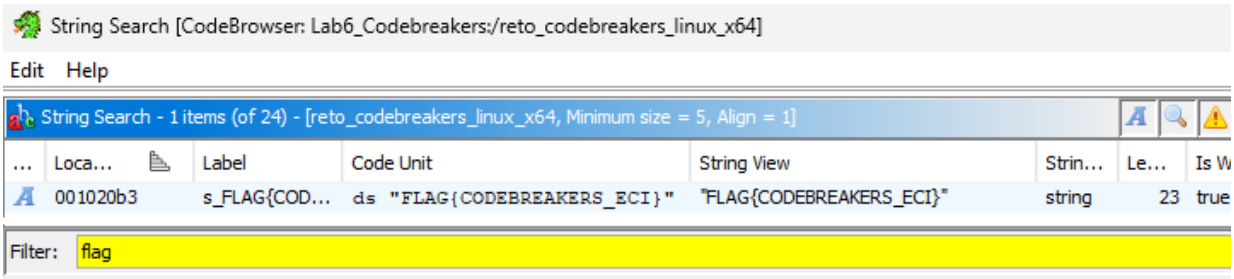


Fig. 1. Búsqueda de cadenas en Ghidra con el filtro «flag»: revela la estructura de la bandera esperada.

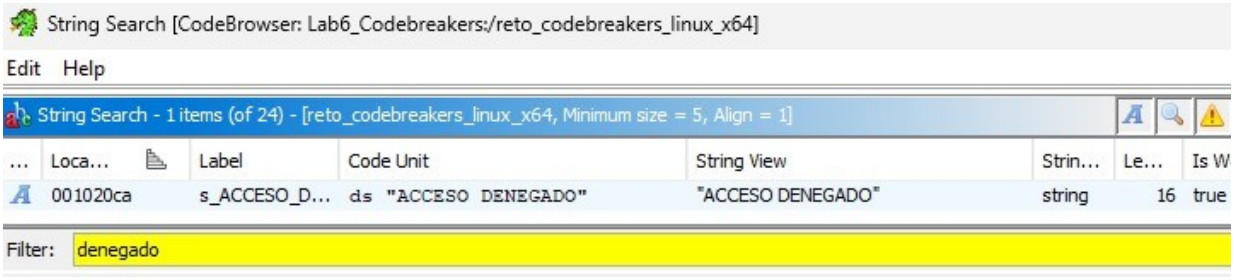


Fig. 2. Búsqueda de cadenas con el filtro «denegado»: identifica el mensaje de error «ACCESO DENEGADO».

IV. ANÁLISIS DINÁMICO Y EVASIÓN

A. Pruebas iniciales en Linux

Para contrastar la hipótesis, el binario se trasladó a Kali Linux y se le otorgaron permisos de ejecución (`chmod +x`). Al ingresar la máscara `ECI_VECTOR_2026!` como clave se obtuvo «ACCESO DENEGADO» (Fig. 5).

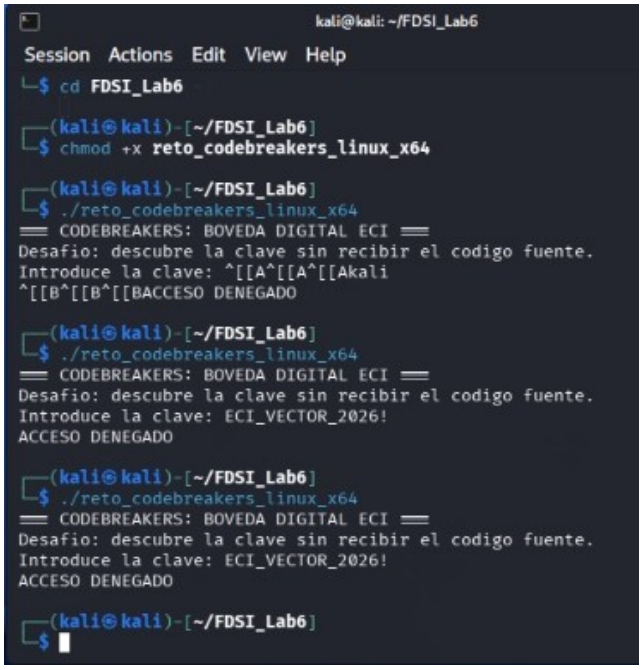


Fig. 5. Primer intento en Kali Linux con la máscara `ECI_VECTOR_2026!` como clave: resultado «ACCESO DENEGADO».

Se sospechó entonces que el salto de línea final alteraba la comparación, por lo que se intentó inyectar la cadena con `echo -n`. Con comillas dobles, el intérprete de comandos procesó el carácter «!» (expansión de historial) y dejó la entrada abierta en el indicador `dquote>` (Fig. 6). Con comillas simples la inyección fue limpia, pero el resultado siguió siendo «ACCESO DENEGADO» (Fig. 7). Esto confirmó en la práctica que `ECI_VECTOR_2026!` era solo un componente del cálculo y no la clave real.

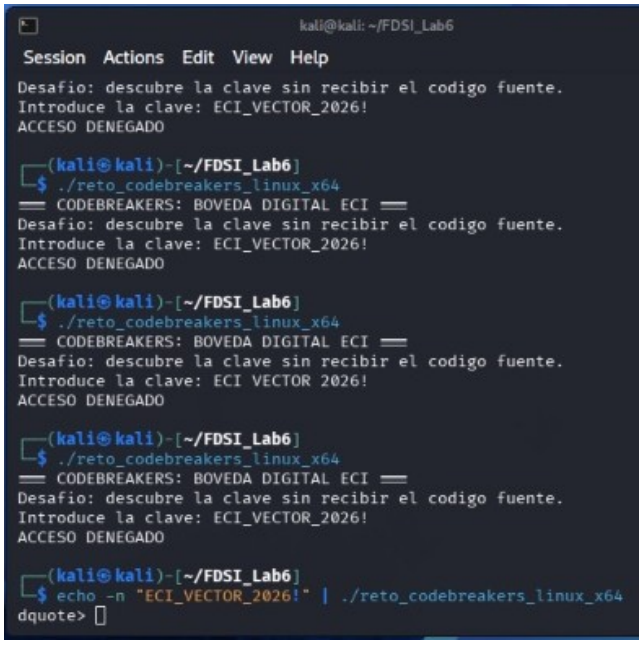


Fig. 6. Inyección con `echo -n` y comillas dobles: el intérprete espera más entrada (`dquote>`) por la expansión del carácter «!».

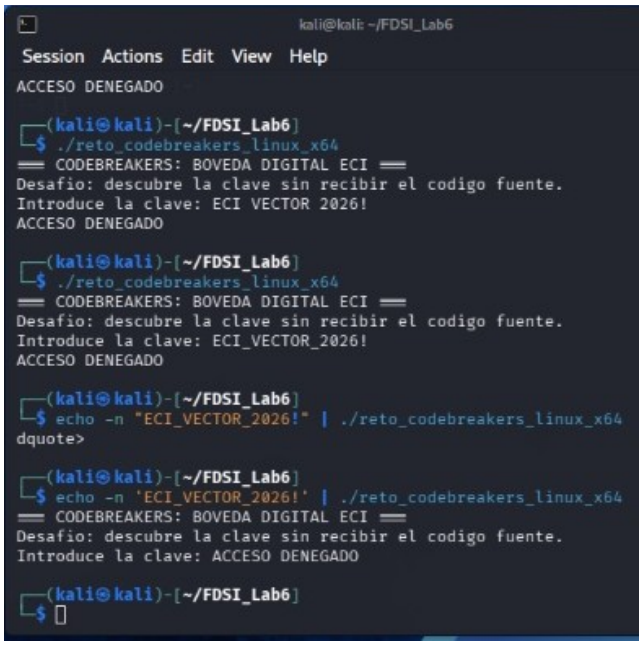


Fig. 7. Inyección con comillas simples: la entrada llega limpia, pero la clave sigue siendo rechazada.

B. Intercepción de memoria con ltrace

Revertir manualmente el cifrado AES implicaba un esfuerzo considerable, por lo que se optó por un enfoque dinámico: se instaló `ltrace` [3] (Fig. 8) para rastrear las llamadas a funciones de biblioteca y capturar la contraseña ya calculada en memoria.

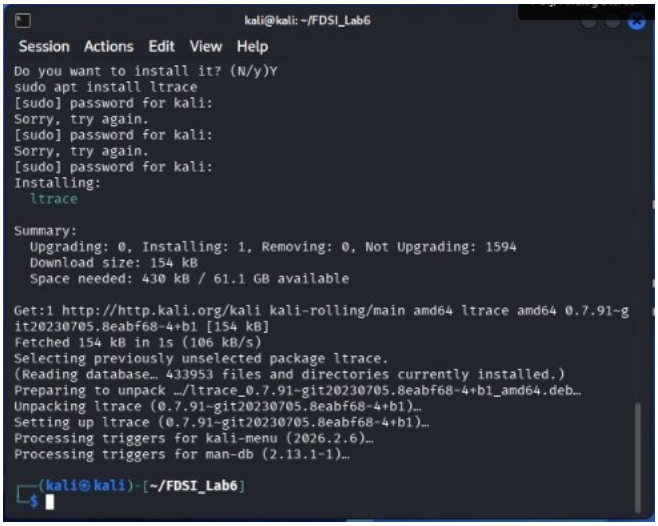


Fig. 8. Instalación de `ltrace` en Kali Linux.

Al ejecutar `ltrace ./reto_codebreakers_linux_x64` e ingresar una cadena arbitraria («AAAA»), la traza muestra que el programa lee la entrada con `fgets`, elimina el salto de línea con `strcspn` y luego invoca `strcmp("AAAA", "BREAKER#2026")`, que devuelve −1 (Fig. 9). El segundo argumento de `strcmp` es la clave ya descifrada, expuesta en texto plano. La llamada a `strcspn` también descarta la sospecha anterior: el salto de línea no era el problema.

V. RESULTADOS

Con la clave `BREAKER#2026` se ejecutó el binario de forma normal, sin `ltrace`. El sistema validó la entrada, mostró «ACCESO CONCEDIDO» y entregó la bandera (Fig. 10). La Tabla II resume las entradas probadas durante el laboratorio.

TABLA II
ENTRADAS PROBADAS Y RESULTADO

Entrada	Método	Resultado
`ECI_VECTOR_2026!`	Entrada interactiva (Fig. 5)	ACCESO DENEGADO
`ECI_VECTOR_2026!`	`echo -n` con comillas simples (Fig. 7)	ACCESO DENEGADO
`AAAA`	Bajo `ltrace` (Fig. 9)	`strcmp` = −1; ACCESO DENEGADO
`BREAKER#2026`	Entrada interactiva (Fig. 10)	ACCESO CONCEDIDO y bandera

Clave descubierta: `BREAKER#2026`. **Bandera obtenida:** `FLAG{CODEBREAKERS_ECI}`.

VI. CONCLUSIONES

El laboratorio demostró los riesgos de ofuscar contraseñas o procesos de validación dentro de un binario del lado del cliente. Aunque el programa empleaba rutinas criptográficas robustas (AES-256) y operaciones a nivel de bits (XOR), el uso de `strcmp`, una función estándar que compara texto plano, expuso el secreto en tiempo de ejecución.

El análisis estático permitió comprender la arquitectura del programa y descartar la máscara como clave, y el análisis dinámico permitió extraer el valor real sin revertir el cifrado. La combinación de ambos resultó altamente eficaz para este reto. Como medida de mitigación, los secretos no deberían protegerse mediante validaciones locales: conviene validar en el servidor y comparar valores derivados (por ejemplo, *hashes* con sal) en lugar de texto plano, sin olvidar que ninguna técnica del lado del cliente es infalible.

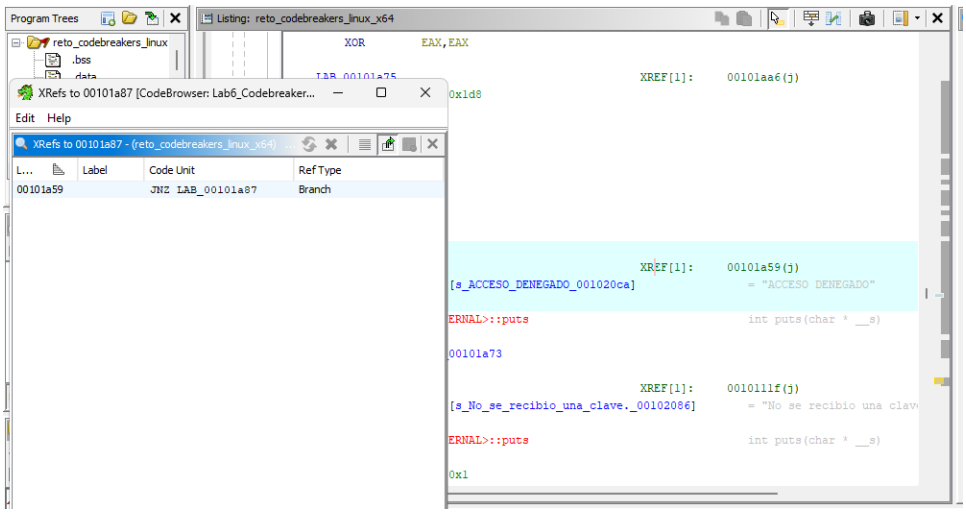


Fig. 3. Referencias cruzadas (XREF) de la cadena «ACCESO DENEGADO»: conducen a la función `main`.

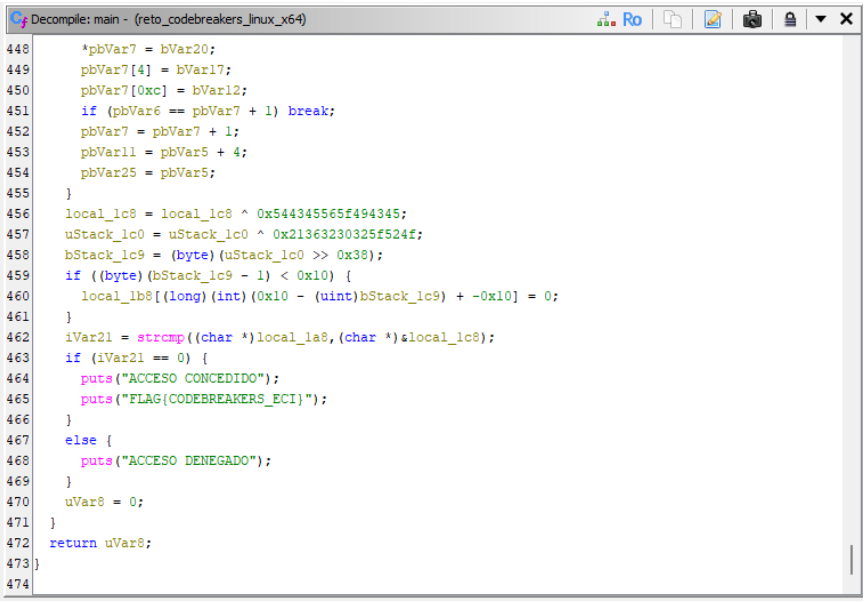


Fig. 4. Vista *Decompiler* de `main`: operaciones XOR con constantes de 64 bits y comparación con `strcmp`.

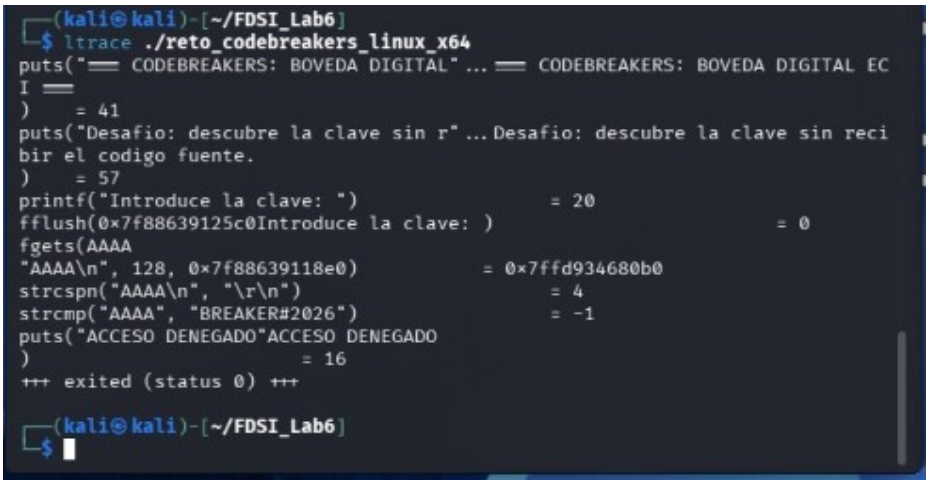


Fig. 9. Traza de `ltrace`: la llamada a `strcmp` expone la clave real, `BREAKER#2026`.

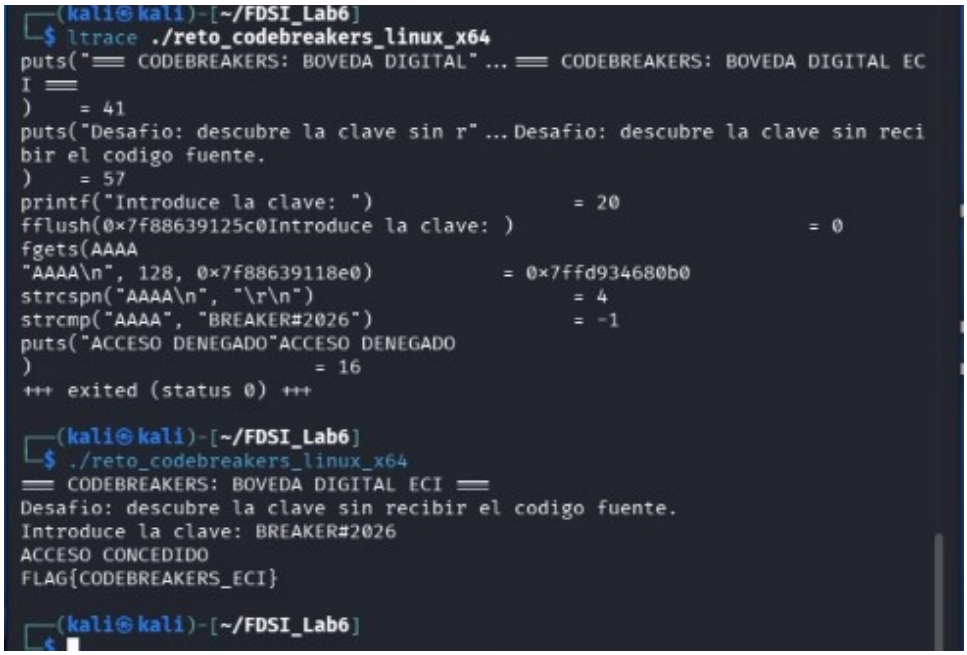


Fig. 10. Ejecución normal con la clave extraída: «ACCESO CONCEDIDO» y obtención de la bandera.

REFERENCIAS

[1] National Security Agency, «Ghidra Software Reverse Engineering Framework». [En línea]. Disponible en: https://ghidra-sre.org/

[2] «Kali Linux». [En línea]. Disponible en: https://www.kali.org/

[3] «ltrace: a library call tracer». [En línea]. Disponible en: https://www.ltrace.org/

[4] «strcmp(3) — Linux manual page», The Linux man-pages project. [En línea]. Disponible en: https://man7.org/linux/man-pages/man3/strcmp.3.html